WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS
PROJECT-BASED LEARNING (PBL) REPORT

PROJECT 24

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR EXAMINATION SCHEDULING AND RESULT PROCESSING SYSTEM

Submitted by

Student Name	Kadasi Levin Josewell
Roll Number	25WU0102113
Program / Section	B.Tech AI & ML, Rhinos

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course. The project provided a practical setting in which to apply relational modelling, keys, constraints, SQL queries, data validation and application-to-database connectivity.

I would like to thank the course faculty, whose guidance and course requirements helped shape the project workflow and its documentation. I also acknowledge the official documentation for Python, SQLite and Streamlit used to understand the technologies used in the prototype. The report and application are intended for academic demonstration; the included records are synthetic and should not be interpreted as official university data.

Course faculty name has intentionally been left blank for completion before submission.

UNDERTAKING

I hereby declare that the project entitled “Design and Implementation of a Database Management System for Examination Scheduling and Result Processing System,” developed as ExamFlow, is submitted as part of the Project-Based Learning activity for the Database Management Systems course. I undertake that the report describes the design and prototype to the best of my knowledge, and that external tools and documentation are acknowledged where used. I understand that the final submission must accurately represent the code, tests performed, screenshots and limitations of the submitted version.

Name: Kadasi Levin Josewell
Programme: B.Tech — Artificial Intelligence and Machine Learning
Section: Rhinos
Roll Number: 25WU0102113
Signature: _____
Date: _____

TABLE OF CONTENTS

Acknowledgement 2
Undertaking 3
List of Tables 5
List of Figures 6
List of Acronyms 7
Abstract 8
CHAPTER 1 PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM 9
1.1 Problem Statement 9
1.2 Objectives 9
1.3 Scope and Assumptions 10
1.4 Proposed Solution Strategy 10
1.5 Functional Requirements 10
1.6 Entity–Relationship Diagram 11
1.7 Entity and Relationship Summary 11
CHAPTER 2 LOGICAL DATABASE DESIGN 13
2.1 Relational Schema 13
2.2 Keys and Integrity Constraints 14
2.3 Functional Dependencies 14
2.4 Normalization Assessment up to 3NF 15
2.5 Data Dictionary 15
2.6 DDL and Sample Data 16
CHAPTER 3 APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN 18
3.1 Application Overview 18
3.2 Development Environment and Database Connectivity 18
3.3 Application Modules and Database Mapping 19
3.4 CRUD Operations 19
3.5 Business Rules, Validation and Error Handling 19
3.6 Reports and Database Operations 20
3.7 Application Screens 20
3.8 Testing and Results 24
CHAPTER 4 CONCLUSION 26
4.1 Limitations and Future Enhancements 26
References 27
Appendices 28

LIST OF TABLES

- Table 1. Scope and Assumptions
- Table 2. Proposed Solution Strategy
- Table 3. Functional Requirements
- Table 4. Entity and Relationship Summary
- Table 5. Relational Schema
- Table 6. Keys and Integrity Constraints
- Table 7. Normalization Assessment
- Table 8. Data Dictionary
- Table 9. Database Implementation Environment
- Table 10. Application Development Environment
- Table 11. Application Module-to-Database Mapping
- Table 12. CRUD Operations
- Table 13. Business Rules, Validation and Error Handling
- Table 14. Reports and Database Operations
- Table 15. Test Cases

LIST OF FIGURES

- Figure 1. ER diagram based on the implemented SQLite schema
- Figure 2. Dashboard overview
- Figure 3. Student records
- Figure 4. Faculty and invigilators
- Figure 5. Subject catalogue
- Figure 6. Examination rooms
- Figure 7. Examination timetable
- Figure 8. Marks entry and result processing
- Figure 9. SQL reports and exports
- Figure 10. Read-only database explorer

LIST OF ACRONYMS

Acronym	Expansion
CRUD	Create, Read, Update and Delete
DBMS	Database Management System
ER / ERD	Entity–Relationship / Entity–Relationship Diagram
FK	Foreign Key
GUI	Graphical User Interface
PK	Primary Key
SQL	Structured Query Language
SQLite	Embedded relational database engine
UI	User Interface
UK	Unique Key / UNIQUE constraint
3NF	Third Normal Form

ABSTRACT

Examination administration involves maintaining student and course records, registering students for examination sessions, allocating rooms and invigilators, recording marks, and preparing results and reports. When these activities are managed using disconnected spreadsheets or manual records, duplicate entries, inconsistent updates, timetable conflicts and delayed reporting can occur. This project presents ExamFlow, an academic prototype of a database-driven Examination Scheduling and Result Processing System. The prototype uses Python with Streamlit for the user interface and SQLite for persistent local storage. Its relational design separates programmes, students, courses, faculty, invigilators, rooms, exam sessions, registrations, schedules, marks and grade rules. Primary keys, foreign keys, uniqueness rules, check constraints and application-level validation are used to protect data integrity. The application provides record-management screens, exam scheduling with date, capacity and overlap checks, marks entry and correction, derived percentage/grade/pass-fail results, SQL-based reports, CSV exports and a read-only query explorer. Screenshots supplied with the project demonstrate the interface and its main workflows, while the source package provides the implementation reference. The dataset is synthetic and the application is a local prototype, not a production university system. The report identifies validation and schema-alignment checks that should be completed before deployment or final evaluation.

Keywords: Examination scheduling, relational database, SQLite, Streamlit, data integrity, result processing

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

An examination cycle requires multiple related records: students and their programmes, courses, faculty and invigilators, rooms and capacities, examination sessions, student registrations, timetable allocations, marks, grades and reports. If these records are kept in separate spreadsheets or updated manually, the same information may be repeated in multiple places. Repeated data increases the chance of inconsistent values and makes it difficult to enforce rules such as unique student registrations, valid exam time intervals, room-capacity limits and the prevention of room or invigilator double-booking.

ExamFlow addresses this problem by bringing the core workflow into one database-backed application. It stores related data in separate tables connected through primary and foreign keys, validates selected scheduling and marks-entry rules, and derives results through a database view. The project is scoped as a demonstrable academic prototype using synthetic records; it does not claim to replace a production student information system or the university's official examination process.

1.2 Objectives

1. Design a relational data model for programmes, students, courses, faculty, rooms, sessions, registrations, schedules and marks.
2. Implement persistent CRUD workflows using Python, Streamlit and SQLite.
3. Validate exam scheduling against session dates, time ordering, room capacity and overlapping room/invigilator assignments.
4. Calculate percentage, grade and pass/fail status from recorded marks and course maximum marks.
5. Generate timetable, registration, performance, pass/fail and room-utilisation reports with CSV export.
6. Document implementation evidence, tests and limitations so that the prototype can be reviewed and reproduced.

1.3 Scope and Assumptions

Category	Project-specific definition
In scope	Master records for programmes, students, courses, faculty, invigilators, rooms and exam sessions; registrations; timetable scheduling; marks entry/correction; result derivation; reports and CSV downloads; read-only SQL exploration.
Out of scope	University-wide integration, online student portal, authentication and role-based access, audit trail, notifications, payment, biometric attendance, cloud/high-availability deployment, formal approval/publishing workflow.

Category	Project-specific definition
Assumptions	Each student has a unique roll number; course codes and room codes are unique; a student may register once for a given course and session; each course has a positive maximum mark value; records used for demonstrations are synthetic.
Data and policy assumptions	Grade boundaries and the 40% pass threshold are illustrative prototype rules. They must be checked against the course faculty's approved grading policy before real use.
Environment	The packaged code creates a local SQLite database file on first launch. A local file database is appropriate for a single-user academic demonstration, not a substitute for production access control or concurrent multi-user operations.

1.4 Proposed Solution Strategy

The solution follows a data-first approach: identify entities and relationships, map them to relations, define keys and constraints, create the local schema, then build application pages that read and write the same database. The user selects a module from the Streamlit navigation, enters or reviews records, and receives a success message or a validation error. SQL joins combine normalized records for timetable and result displays.

Solution element	Project-specific description
Primary users / roles	Academic administration staff or a demonstration user maintaining examination data. The current prototype does not implement distinct authenticated roles.
Major modules	Dashboard; Students; Courses; Faculty & Invigilators; Rooms; Exam Sessions; Registrations; Exam Scheduling; Marks & Results; Reports; Database Explorer.
Workflow	Create master records → create an exam session → register students for courses → schedule exams → enter/correct marks → review derived results → export reports.
Business rules	Unique identifiers; foreign-key references; valid date range and time order; room capacity; no overlapping room/invigilator allocation; one registration per student/course/session; one mark record per registration.
Outputs	Dashboard metrics, exam timetable, results register, course registration counts, course performance, pass/fail summary, room utilisation and CSV files.

1.5 Functional Requirements

Req. ID	Functional requirement	Priority / status
FR-01	Display summary counts and recent registration/result information from the database.	Core; screen evidence supplied.
FR-02	Create, list, update and conditionally remove student/course/faculty/room/session records.	Core; verify each CRUD path before final submission.
FR-03	Register a student for a course in an examination session without duplicate registration.	Core; UNIQUE constraint in schema.
FR-04	Schedule a course examination only when the date/time and resource checks pass.	Core; validation logic exists in packaged app.py.
FR-05	Record and correct marks for a registration.	Core; screen evidence supplied.
FR-06	Derive percentage, grade and result status from marks and maximum marks.	Core; result_view in schema.
FR-07	Display SQL reports and export report/result data as CSV.	Core; screen evidence supplied.
FR-08	Browse selected tables and execute read-only SELECT/WITH queries.	Core; query screen evidence supplied; read-only enforcement should be strengthened.

1.6 Entity–Relationship Diagram (ERD)

Figure 1 represents the entity and key relationships found in the packaged app.py schema. It intentionally uses the implementation names (programmes, courses, exam_sessions, registrations, schedules and marks), rather than the alternate labels used in some review slides. A course belongs to a programme; registrations link students, courses and sessions; schedules link a session/course to a room and invigilator; marks are associated with a registration. The result_view is a derived view rather than a base entity.

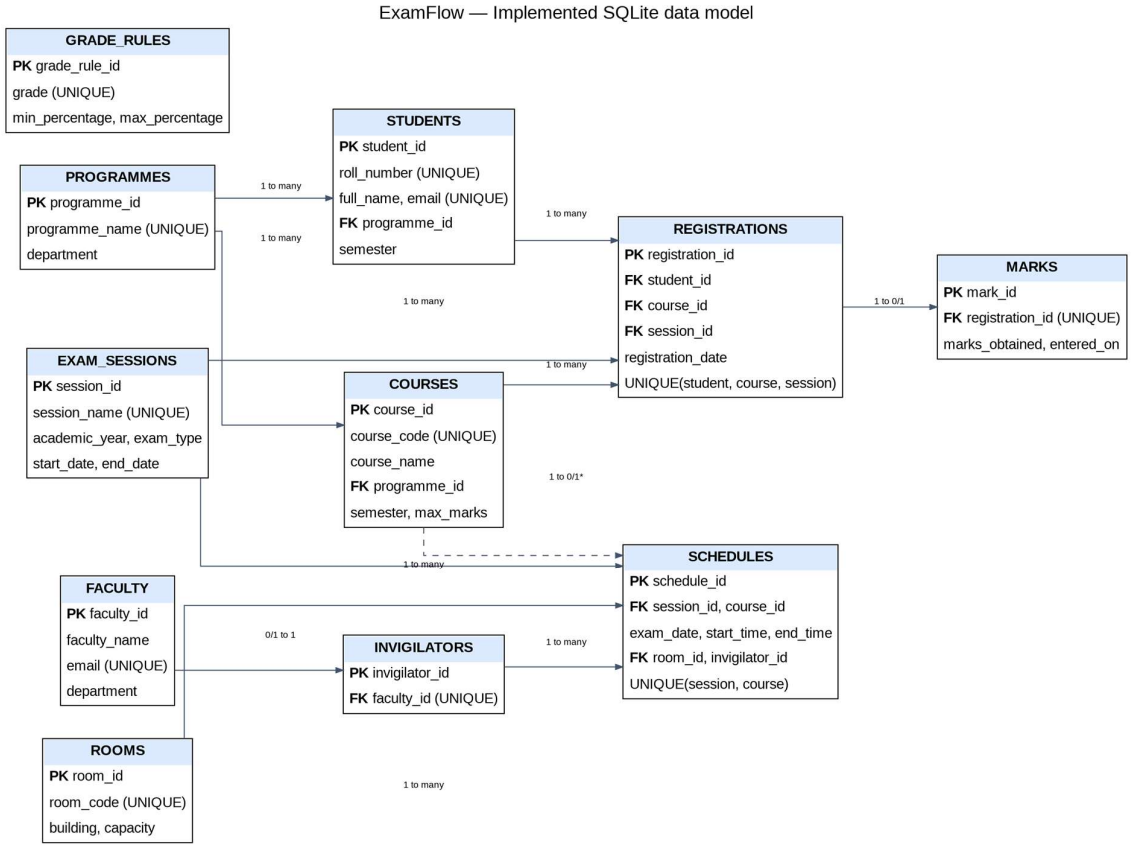


*Figure 1. ER diagram based on the implemented SQLite schema. *The UNIQUE(session_id, course_id) constraint permits one schedule per course per session.*

1.7 Entity and Relationship Summary

Entity / relationship	Key attributes / cardinality	Purpose / rule
Programmes	programme_id PK; programme_name UNIQUE; 1 programme : many students/courses	Stores academic programme and department.
Students	student_id PK; roll_number UNIQUE; programme_id FK	Stores student identity and semester.
Courses	course_id PK; course_code UNIQUE; programme_id FK	Stores course catalogue, semester and maximum marks.
Faculty / Invigilators	faculty_id PK; invigilator_id PK; faculty_id UNIQUE FK	Separates faculty contact data from invigilation eligibility.
Rooms	room_id PK; room_code UNIQUE	Stores location and positive seating capacity.
Exam sessions	session_id PK; session_name UNIQUE	Defines academic year, type and allowed date range.
Registrations	registration_id PK; FKs to student/course/session; composite uniqueness	Associative relation prevents duplicate student-course-session entries.

Entity / relationship	Key attributes / cardinality	Purpose / rule
Schedules	schedule_id PK; FKs to session/course/room/invigilator	Stores timetable allocation; overlap checks are application-level.
Marks	mark_id PK; registration_id UNIQUE FK	Stores one marks record per registration.
Grade rules	grade_rule_id PK; grade UNIQUE	Stores percentage bands, but the current result view uses hard-coded CASE thresholds; see limitations.
result_view	Derived view; joins registrations, students, courses, sessions and marks	Calculates percentage, grade and Pending/Pass/Fail status.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The packaged implementation defines eleven base tables and one view. The schema is created in init_db() inside app.py when the application starts. The separate schema.sql file in the provided ZIP is only a reference note and does not contain the full DDL; Appendix A therefore identifies app.py as the authoritative schema source for this package.

- PROGRAMMES(programme_id PK, programme_name UNIQUE NOT NULL, department NOT NULL)
- STUDENTS(student_id PK, roll_number UNIQUE NOT NULL, full_name NOT NULL, email UNIQUE, programme_id FK NOT NULL, semester CHECK 1–12)
- COURSES(course_id PK, course_code UNIQUE NOT NULL, course_name NOT NULL, programme_id FK NOT NULL, semester CHECK 1–12, max_marks CHECK > 0)
- FACULTY(faculty_id PK, faculty_name NOT NULL, email UNIQUE NOT NULL, department NOT NULL)
- ROOMS(room_id PK, room_code UNIQUE NOT NULL, building NOT NULL, capacity CHECK > 0)
- EXAM_SESSIONS(session_id PK, session_name UNIQUE NOT NULL, academic_year NOT NULL, exam_type NOT NULL, start_date NOT NULL, end_date NOT NULL, end_date >= start_date)
- INVIGILATORS(invigilator_id PK, faculty_id UNIQUE FK NOT NULL)
- REGISTRATIONS(registration_id PK, student_id FK, course_id FK, session_id FK, registration_date DEFAULT current date, UNIQUE(student_id, course_id, session_id))
- SCHEDULES(schedule_id PK, session_id FK, course_id FK, exam_date, start_time, end_time, room_id FK, invigilator_id FK, UNIQUE(session_id, course_id), end_time > start_time)
- MARKS(mark_id PK, registration_id UNIQUE FK, marks_obtained CHECK >= 0, entered_on DEFAULT current timestamp)
- GRADE_RULES(grade_rule_id PK, grade UNIQUE, min_percentage, max_percentage, CHECK 0 <= min <= max <= 100)

The view result_view joins registrations with students, courses, sessions and marks. It calculates percentage as marks_obtained × 100 / max_marks, rounds the display to two decimal places,

assigns grade bands (A+, A, B, C, D, F), and returns Pending when no mark is entered. The view labels a percentage of 40 or more as Pass and lower percentages as Fail.

2.2 Keys and Integrity Constraints

Constraint type	Implemented example	Effect / notes
Primary key	Integer surrogate IDs on every base table	Identifies each row and supports stable foreign-key references.
Foreign key	students.programme_id → programmes.programme_id	Prevents references to missing parent rows when foreign-key enforcement is enabled by the connection.
UNIQUE	students.roll_number; courses.course_code; rooms.room_code	Prevents duplicate natural identifiers.
Composite UNIQUE	registrations(student_id, course_id, session_id)	Prevents duplicate registrations for the same student/course/session.
NOT NULL	Required names, codes, date fields and foreign keys	Requires key operational values.
CHECK	semester BETWEEN 1 AND 12; capacity > 0; max_marks > 0	Restricts values to plausible ranges.
CHECK time/date	end_date >= start_date; end_time > start_time	Prevents invalid session windows and zero/negative schedule intervals.
Marks constraints	marks_obtained >= 0 and UI input max is course.max_marks	The table-level CHECK shown in app.py does not itself enforce marks_obtained <= max_marks; the form bounds the entry. Add a stronger database-level rule/trigger before production use.
Application-level scheduling	Checks session date, capacity and overlapping room/invigilator allocations	Provides useful error messages but is not a database exclusion constraint; concurrent writes would require stronger transactional enforcement.

2.3 Functional Dependencies

The principal functional dependencies follow from the primary and unique keys. These dependencies are used to reason about duplication and decomposition. They assume the intended uniqueness rules are maintained.

Relation	Representative functional dependency
PROGRAMMES	programme_id → programme_name, department; programme_name → programme_id, department
STUDENTS	student_id → roll_number, full_name, email, programme_id, semester; roll_number → student_id, full_name, email, programme_id, semester
COURSES	course_id → course_code, course_name, programme_id, semester, max_marks; course_code → remaining course attributes
FACULTY	faculty_id → faculty_name, email, department; email → faculty_id, faculty_name, department
ROOMS	room_id → room_code, building, capacity; room_code → room_id, building, capacity
EXAM_SESSIONS	session_id → session_name, academic_year, exam_type, start_date, end_date; session_name → remaining attributes
REGISTRATIONS	registration_id → student_id, course_id, session_id, registration_date; (student_id, course_id, session_id) → registration_id, registration_date
SCHEDULES	schedule_id → session_id, course_id, exam_date, start_time, end_time, room_id, invigilator_id; (session_id, course_id) → schedule attributes
MARKS	mark_id → registration_id, marks_obtained, entered_on; registration_id → mark attributes
GRADE_RULES	grade_rule_id → grade, min_percentage, max_percentage; grade → grade_rule_id, min_percentage, max_percentage

2.4 Normalization Assessment up to 3NF

The design separates facts with different lifecycles into distinct relations. Student details are not repeated in each result row; course attributes are stored once in COURSES; room details are stored once in ROOMS; and REGISTRATIONS resolves the many-to-many relationship among students, courses and exam sessions. This structure reduces update and insertion anomalies compared with a single wide table.

The intended design is in 3NF where every non-key attribute in each base relation depends on a key, the whole key and nothing but the key. This is a design assessment, not a formal proof for every possible business rule. In particular, the grade-rule relation is not currently used by result_view, and schedule-conflict checks are implemented in application code rather than by a declarative relational constraint.

Relation group	1NF	2NF	3NF assessment	Reason / action
Programmes, students, courses, faculty, rooms, sessions	Yes by design	Yes; single-column surrogate PKs	Expected to satisfy 3NF under stated dependencies	Attributes are atomic and descriptive facts are kept with their entity.
Invigilators	Yes	Yes	Expected to satisfy 3NF	One unique faculty reference per invigilator row.
Registrations	Yes	Yes; composite uniqueness identifies a registration	Expected to satisfy 3NF	Relationship facts are isolated; registration date depends on the registration tuple.
Schedules	Yes	Yes; schedule_id and session/course uniqueness	Expected to satisfy 3NF	Time and resource assignment are stored in one schedule relation.
Marks	Yes	Yes	Expected to satisfy 3NF	One marks row per registration; marks facts do not duplicate student/course details.
Grade rules	Yes	Yes	Expected to satisfy 3NF	Grade maps to its percentage band; verify ranges are non-overlapping.
result_view	Derived view	Not assessed as a stored base relation	Derived output, not normalized storage	Contains joined display attributes and computed expressions by design.

2.5 Data Dictionary

Table	Attribute(s)	Type	Key / constraint	Description
programmes	programme_id	INTEGER	PK, auto-increment	Unique programme identifier.
programmes	programme_name	TEXT	NOT NULL, UNIQUE	Programme title.
programmes	department	TEXT	NOT NULL	Owning department/school.
students	student_id	INTEGER	PK, auto-increment	Unique student row ID.
students	roll_number	TEXT	NOT NULL, UNIQUE	Institutional-style demo roll number.
students	full_name	TEXT	NOT NULL	Student name.
students	email	TEXT	UNIQUE, nullable	Optional contact email.
students	programme_id	INTEGER	FK, NOT NULL	Programme reference.
students	semester	INTEGER	CHECK 1–12	Current semester.
courses	course_id	INTEGER	PK, auto-increment	Unique course row ID.
courses	course_code	TEXT	NOT NULL, UNIQUE	Course code.
courses	course_name	TEXT	NOT NULL	Course title.
courses	programme_id	INTEGER	FK, NOT NULL	Owning programme.

Table	Attribute(s)	Type	Key / constraint	Description
courses	semester	INTEGER	CHECK 1–12	Semester associated with the course.
courses	max_marks	INTEGER	DEFAULT 100, CHECK > 0	Maximum obtainable marks.
faculty	faculty_id	INTEGER	PK, auto-increment	Unique faculty row ID.
faculty	faculty_name	TEXT	NOT NULL	Faculty name.
faculty	email	TEXT	NOT NULL, UNIQUE	Faculty email.
faculty	department	TEXT	NOT NULL	Faculty department.
rooms	room_id	INTEGER	PK, auto-increment	Unique room row ID.
rooms	room_code	TEXT	NOT NULL, UNIQUE	Room identifier.
rooms	building	TEXT	NOT NULL	Building/location.
rooms	capacity	INTEGER	CHECK > 0	Maximum room capacity.
exam_sessions	session_id	INTEGER	PK, auto-increment	Unique examination session.
exam_sessions	session_name	TEXT	NOT NULL, UNIQUE	Session label.
exam_sessions	academic_year	TEXT	NOT NULL	Academic year label.
exam_sessions	exam_type	TEXT	NOT NULL	Exam type.
exam_sessions	start_date / end_date	TEXT	NOT NULL; end >= start	Allowed session date range.
invigilators	invigilator_id	INTEGER	PK, auto-increment	Unique invigilator row.
invigilators	faculty_id	INTEGER	FK, UNIQUE, NOT NULL	Faculty member assigned as invigilator.
registrations	registration_id	INTEGER	PK, auto-increment	Unique registration.
registrations	student_id / course_id / session_id	INTEGER	FKs; composite UNIQUE	Links student, course and exam session.
registrations	registration_date	TEXT	DEFAULT date(now)	Date registration row was created.
schedules	schedule_id	INTEGER	PK, auto-increment	Unique schedule row.
schedules	session_id / course_id	INTEGER	FKs; composite UNIQUE	Session and course being scheduled.
schedules	exam_date	TEXT	NOT NULL	Calendar date of examination.
schedules	start_time / end_time	TEXT	NOT NULL; end > start	Time interval.
schedules	room_id / invigilator_id	INTEGER	FKs, NOT NULL	Assigned resources.
marks	mark_id	INTEGER	PK, auto-increment	Unique marks row.
marks	registration_id	INTEGER	FK, UNIQUE, NOT NULL	Registration receiving marks.
marks	marks_obtained	REAL	NOT NULL, CHECK >= 0	Marks entered.
marks	entered_on	TEXT	DEFAULT datetime(now)	Timestamp of mark entry/update.
grade_rules	grade_rule_id	INTEGER	PK, auto-increment	Unique grade rule.
grade_rules	grade	TEXT	NOT NULL, UNIQUE	Grade label.
grade_rules	min_percentage / max_percentage	REAL	CHECK 0 <= min <= max <= 100	Configured percentage band; currently not referenced by result_view.

2.6 Implementation of Logical Design: DDL and Sample Data

Item	Details
DBMS	SQLite; database file is created locally beside app.py.
DBMS version	Not pinned in the submitted package; runtime SQLite version depends on the Python installation.
Base tables	11
View	1 (result_view)
Application layer	Python 3.10+ stated in README; Streamlit and pandas are imported by app.py.
Seed data	Small fictional sample set is inserted by init_db() when the database is empty. The screenshots show a larger mock dataset from a separate running version; counts must be reconciled with the packaged source before submission.

Representative DDL, adapted directly from the implemented init_db() definition:

```
CREATE TABLE IF NOT EXISTS students (
  student_id INTEGER PRIMARY KEY AUTOINCREMENT,
  roll_number TEXT NOT NULL UNIQUE,
  full_name TEXT NOT NULL,
  email TEXT UNIQUE,
  programme_id INTEGER NOT NULL,
  semester INTEGER NOT NULL CHECK (semester BETWEEN 1 AND 12),
  FOREIGN KEY (programme_id) REFERENCES programmes(programme_id)
);

CREATE TABLE IF NOT EXISTS registrations (
  registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
  student_id INTEGER NOT NULL,
  course_id INTEGER NOT NULL,
  session_id INTEGER NOT NULL,
  registration_date TEXT NOT NULL DEFAULT (date('now')),
  UNIQUE(student_id, course_id, session_id),
  FOREIGN KEY(student_id) REFERENCES students(student_id),
  FOREIGN KEY(course_id) REFERENCES courses(course_id),
  FOREIGN KEY(session_id) REFERENCES exam_sessions(session_id)
);
```

The complete implemented schema is embedded in app.py:init_db(). The separate schema.sql file in the provided ZIP currently contains only a comment pointing to app.py; replace it with the full executable DDL before presenting schema.sql as the standalone database script.

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

ExamFlow is a local browser-based prototype. Streamlit renders the pages and controls, Python handles workflow logic and parameterized database operations, and SQLite stores the records. Pandas is used to display query results as tables and prepare CSV downloads. A shared connection helper enables foreign-key enforcement on each connection, while query_df() reads SQL results and execute() commits writes. The success_or_error() helper reports successful operations and catches integrity or general exceptions for display.

3.2 Development Environment and Database Connectivity

Item	Details
Programming language	Python.
UI framework	Streamlit.
Database	SQLite using Python sqlite3.
Data display / export	pandas DataFrame display and CSV serialization.
Persistence	Local file exam_system.db, created beside app.py.
Run command	streamlit run app.py after installing requirements.txt.
Repository	https://github.com/levinjosewell/examination-scheduling-result-processing-system

Database connectivity is centralized in get_conn(), which opens a sqlite3 connection, sets row_factory to sqlite3.Row and executes PRAGMA foreign_keys = ON. query_df() calls pandas.read_sql_query() for reads. execute() runs a parameterized statement, commits the transaction and returns the last inserted row ID. User-provided values are passed as SQL parameters in the main record-management workflows.

3.3 Application Modules and Database Mapping

Module / screen	Purpose	Tables / view	Main operations
Dashboard	Show record counts, recent registrations and result distribution.	students, courses, exam_sessions, schedules, registrations, marks, result_view	SELECT, COUNT, JOIN, GROUP BY.
Students	Add, display and update student records.	students, programmes	INSERT, SELECT, UPDATE; controlled delete where implemented.
Courses	Maintain course catalogue and maximum marks.	courses, programmes	INSERT, SELECT, UPDATE / controlled removal.
Faculty & Invigilators	Maintain faculty and invigilation records.	faculty, invigilators	INSERT, SELECT and relational joins.
Rooms	Maintain room code, building and capacity.	rooms	INSERT, SELECT, UPDATE / controlled removal.
Exam Sessions	Maintain session dates, type and academic year.	exam_sessions	INSERT, SELECT and date validation.
Registrations	Link student/course/session and prevent duplicates.	registrations, students, courses, exam_sessions	INSERT, SELECT, UNIQUE constraint.
Exam Scheduling	Allocate a course/session to date/time, room and invigilator.	schedules, exam_sessions, courses, rooms, invigilators, faculty, registrations	SELECT, INSERT, overlap/capacity/date checks.
Marks & Results	Enter and correct marks; view derived result fields.	marks, registrations, result_view	INSERT, UPDATE, SELECT, CSV export.

Module / screen	Purpose	Tables / view	Main operations
Reports	Generate timetable, result, registration, performance, pass/fail and utilisation reports.	Multiple tables and result_view	JOIN, GROUP BY, aggregate functions, CSV export.
Database Explorer	Browse selected tables and run read-only query interface.	Base tables and result_view	SELECT / WITH display.

3.4 CRUD Operations

Operation	Application feature	Tables involved	Implementation note
Create	Add student/course/faculty/room/session, registration, schedule and marks forms.	Relevant base table	Parameterized INSERT statements; UI validates required fields in several forms.
Read / search	Tables, dashboard, timetable, results, reports and explorer.	Base tables and view	SELECT queries; joins used for human-readable outputs.
Update	Student/course/faculty/room records where update forms exist; mark correction.	Relevant base table	UPDATE statements; mark update refreshes entered_on timestamp.
Delete / controlled removal	Where the UI exposes removal for a master-data row.	Relevant base table	Foreign-key integrity may block removal of referenced rows; verify each delete path before claiming full CRUD coverage.

3.5 Business Rules, Validation and Error Handling

Rule	Implementation found in source	Evidence / caution
Required student details	Student form checks non-empty roll number and full name.	Source-level implementation; capture a negative test result.
Duplicate natural identifiers	UNIQUE constraints on roll numbers, course codes, room codes, emails and session names.	Database integrity exception is caught and shown.
Duplicate registration	UNIQUE(student_id, course_id, session_id).	Database-level protection.
Valid session range	Exam date is compared with session start_date and end_date.	Application-level validation.
Room capacity	Registered count for the course/session is compared with selected room capacity.	Application-level check; confirm whether registration count matches the real expected attendance definition.
Overlapping room/invigilator	Existing schedules are queried for same date with start_time < new end and end_time > new start; same room OR same invigilator causes rejection.	Application-level check; user reported testing overlapping room validation. Not a concurrency-safe database constraint.
Time order	Form checks end time is later than start time; schedules table also has CHECK(end_time > start_time).	Both UI and database checks are represented.
Marks bounds	Marks input control sets minimum 0 and maximum equal to course max_marks. Table CHECK only enforces >= 0.	Add database trigger/check strategy to enforce upper bound independently of the UI.
Result calculation	result_view derives percentage, grade and pass/fail status.	Grade thresholds are hard-coded in CASE; grade_rules table is currently not used by the view.
Read-only query explorer	UI rejects statements not beginning with SELECT or WITH.	This prefix test is not a complete SQL security boundary; use a restricted connection/authorizer or remove custom SQL input before production.

3.6 Reports and Database Operations

Report / operation	SQL concept	Output

Report / operation	SQL concept	Output
Exam timetable	JOIN across sessions, courses, schedules, rooms, invigilators and faculty; ORDER BY date/time.	Chronological timetable with room and invigilator.
Student results	SELECT from result_view; ORDER BY roll number/course.	Marks, maximum marks, percentage, grade and result status.
Course registration counts	CROSS JOIN sessions with courses; LEFT JOIN registrations; COUNT and GROUP BY.	Registration count for each course/session combination.
Course-wise performance	AVG, MAX, MIN, COUNT and GROUP BY.	Average, highest and lowest percentage and graded count.
Pass/fail summary	GROUP BY session and result status.	Count of pending/pass/fail results by session.
Room utilisation	LEFT JOIN rooms, schedules, courses and registrations; aggregate counts.	Room, date/time, course and registered count.
CSV exports	DataFrame.to_csv(index=False) and Streamlit download button.	Downloadable CSV for review and analysis.
Database explorer	SELECT * FROM selected table LIMIT 500; custom SELECT/WITH query.	Tabular view of database contents.

3.7 Application Screens

The following figures reproduce the screenshots supplied with the project materials. They demonstrate the visual prototype and are included as evidence of the interface. The supplied screenshots display labels and fields such as Subjects, Exams, Results, sections and publication status that do not exactly match the packaged ZIP's app.py schema names and forms. Before submission, ensure the screenshots and repository contain the same final application version.

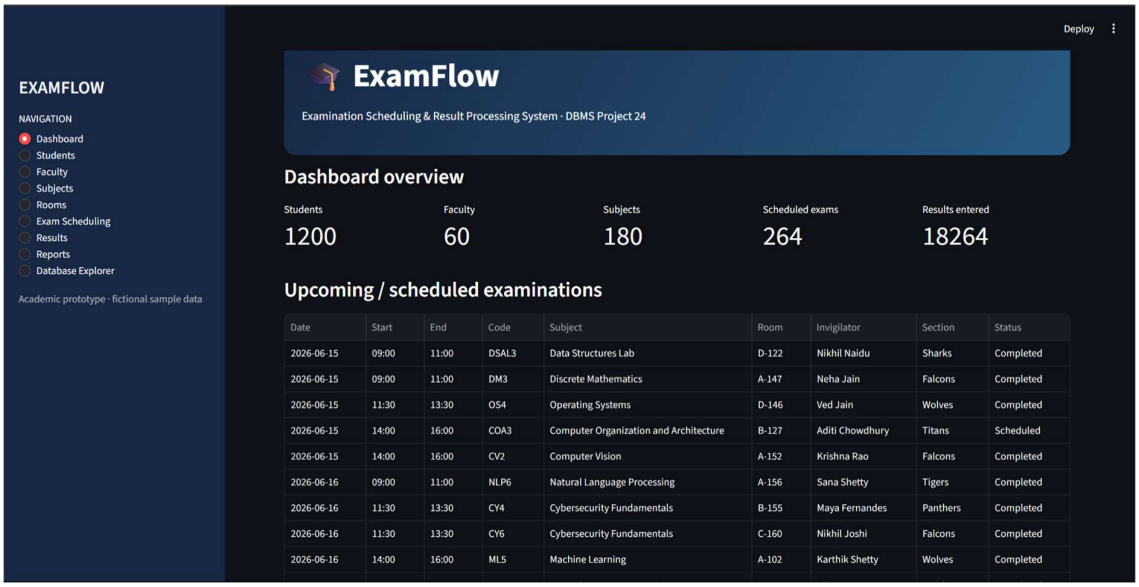


Figure 2. Dashboard overview showing synthetic data counts and examination table.

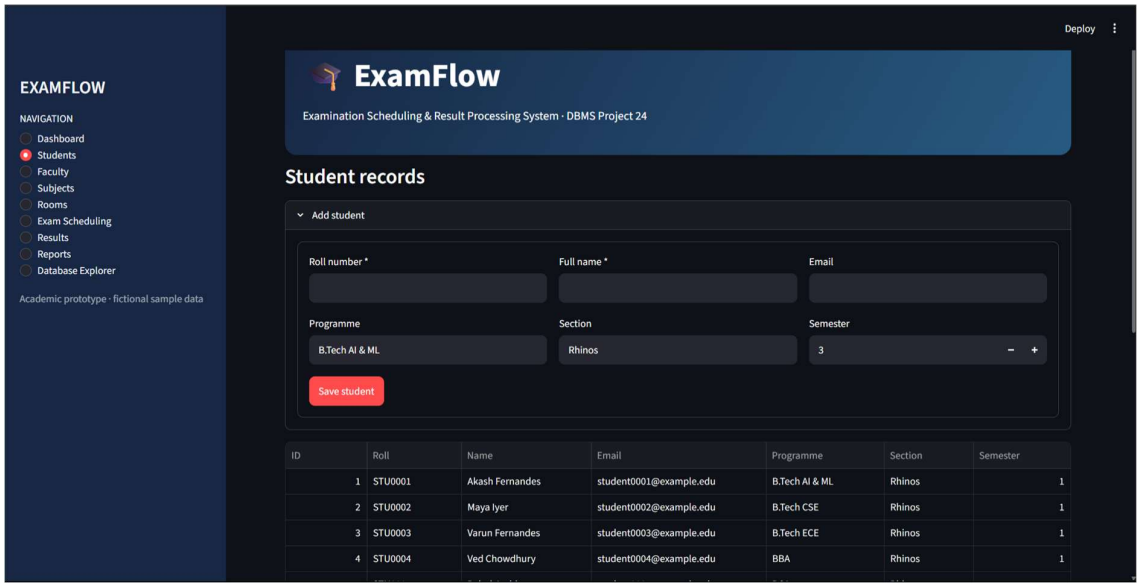


Figure 3. Student records screen with add-student form and table.

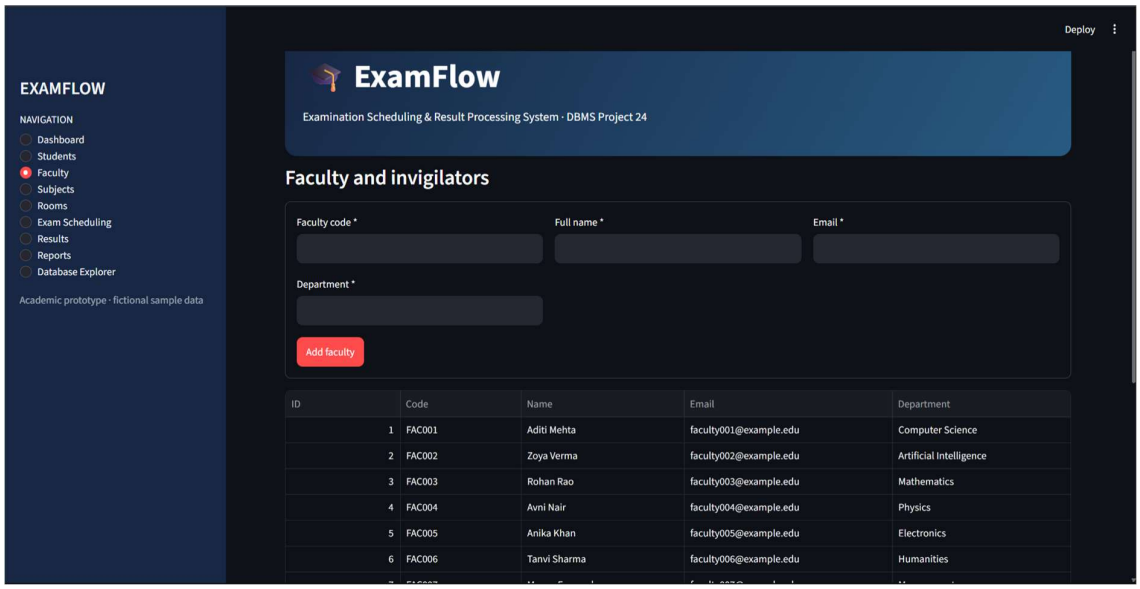


Figure 4. Faculty and invigilators screen.

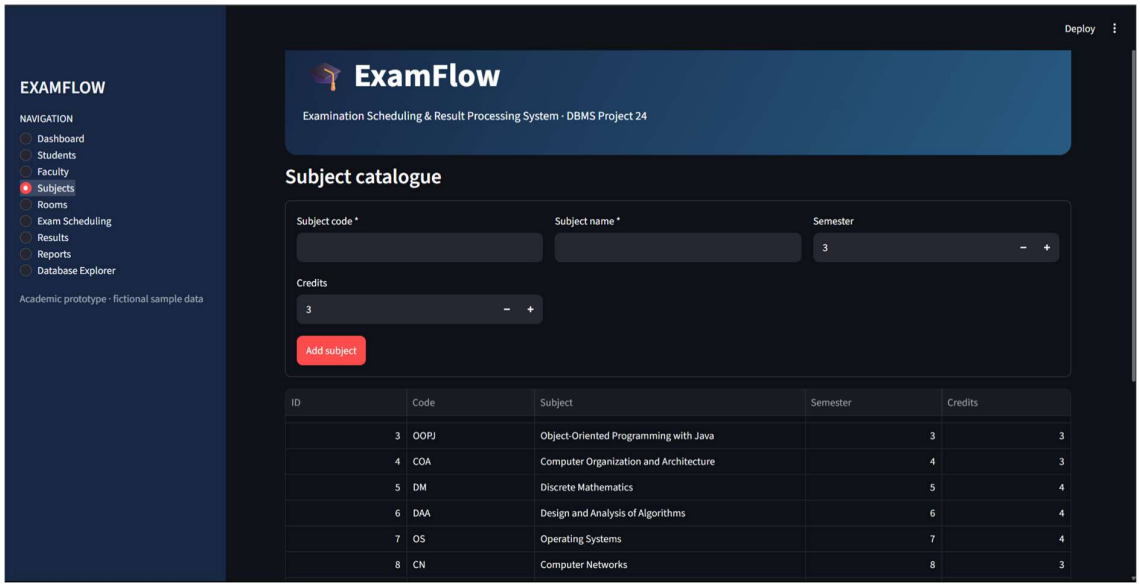


Figure 5. Subject catalogue screen.

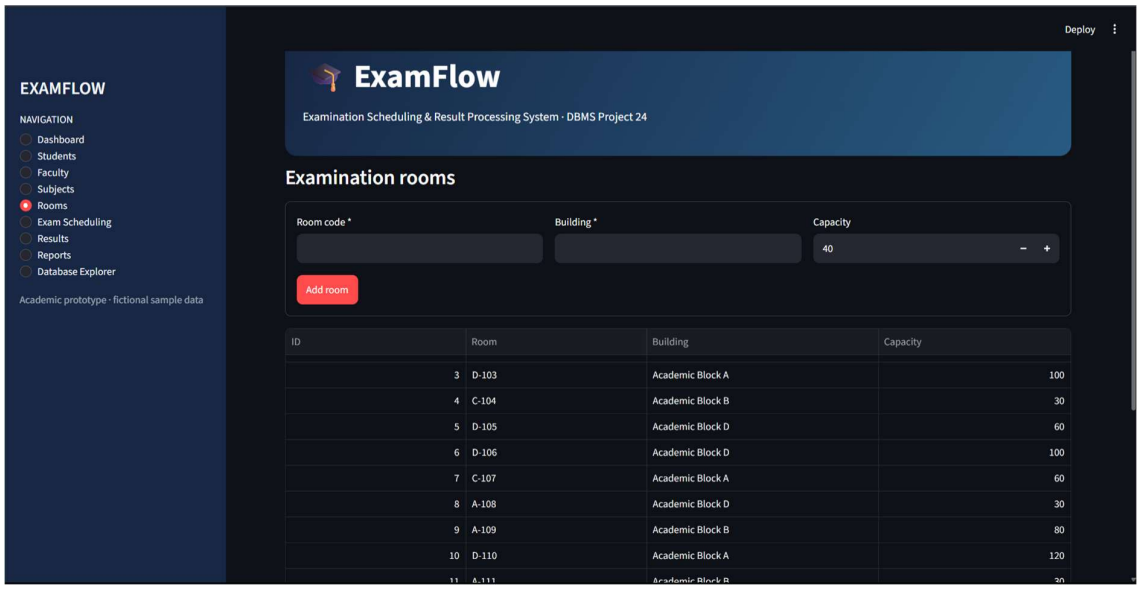


Figure 6. Examination rooms screen.

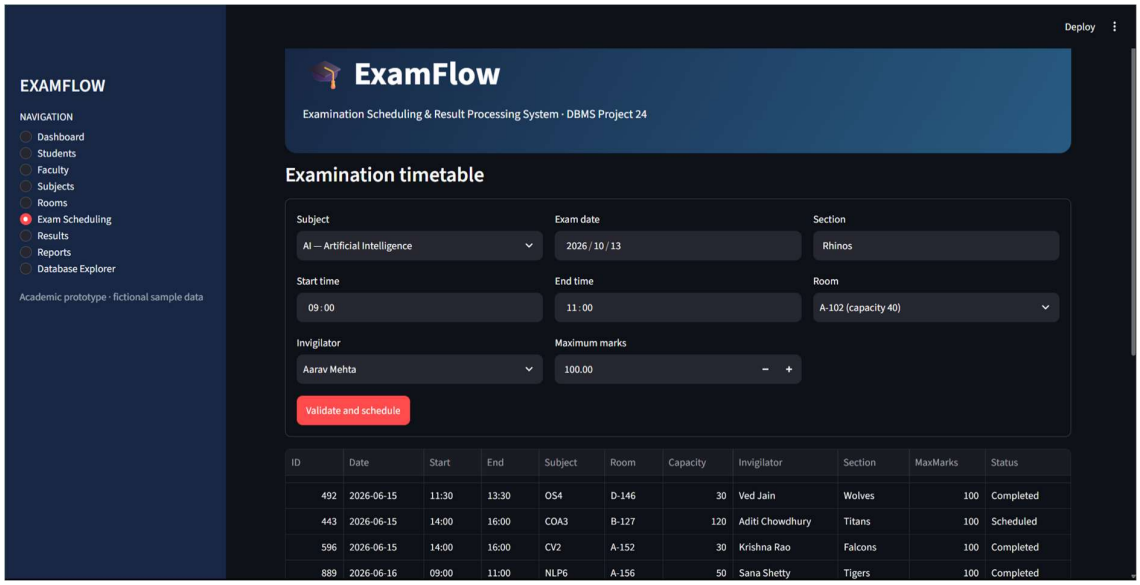


Figure 7. Examination timetable and scheduling form.

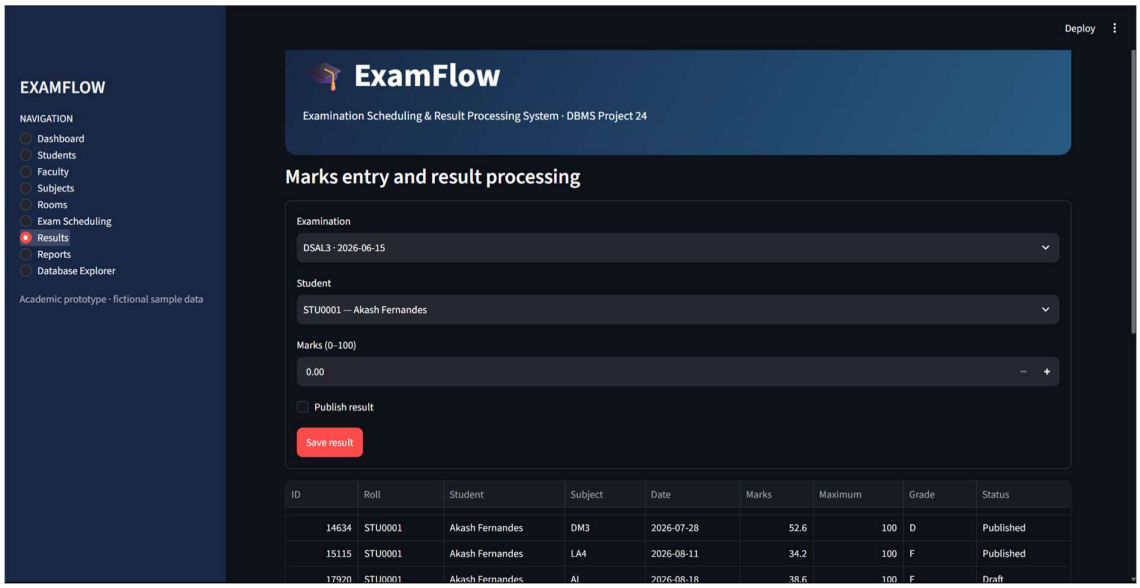


Figure 8. Marks entry and result processing screen.

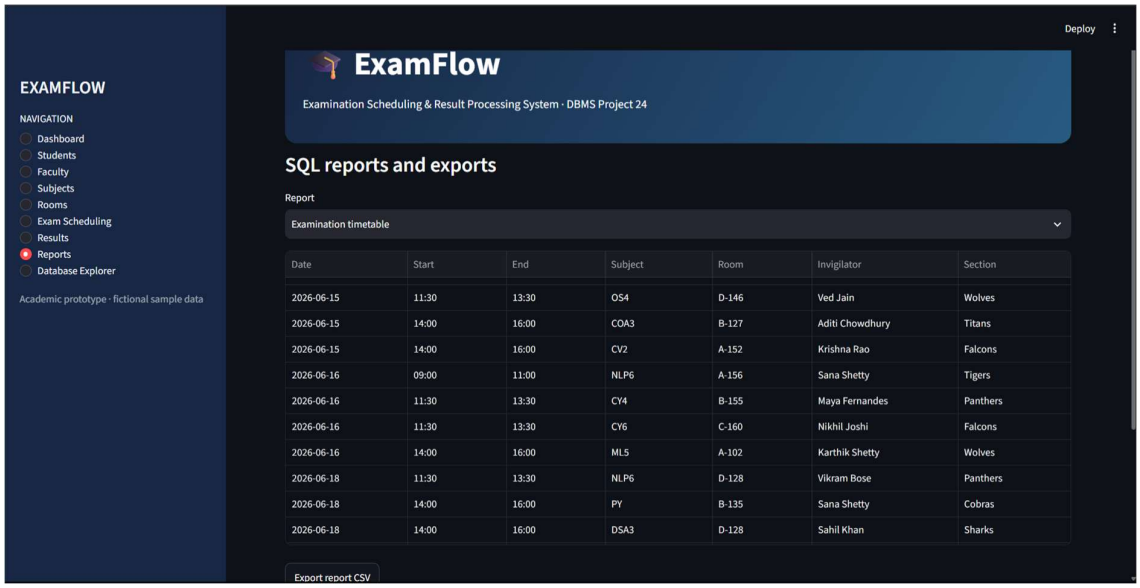


Figure 9. SQL reports and exports screen.

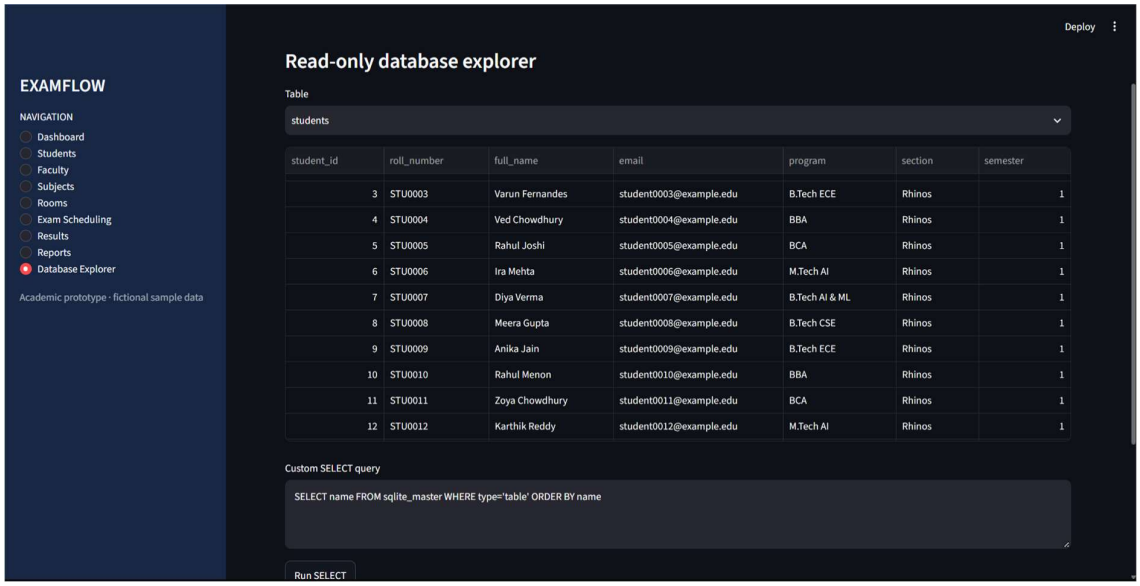


Figure 10. Read-only database explorer screen.

3.8 Testing and Results

The table below distinguishes tests reported by the developer from tests that still require repeatable evidence. “Reported pass” means the user previously stated that the workflow was tested; it is not an independently rerun test in this report. “Pending” must be changed only after the packaged version has been tested and the actual output recorded.

Test ID	Feature / action	Expected result	Actual result / evidence	Status
TC-01	Launch app and inspect dashboard.	Application loads and reads counts from local database.	Screenshots show a dashboard; exact count values differ from the small seed set in packaged app.py.	Reported / reconcile version

Test ID	Feature / action	Expected result	Actual result / evidence	Status
TC-02	Add a student and refresh/reopen.	Record persists and appears in the student list.	User reported persistence test; supplied screenshot shows student form/list.	Reported pass
TC-03	Add or inspect course/subject compatibility.	Course is linked to the intended programme/semester.	User reported subject compatibility test; packaged code uses courses/programmes, while screenshots show a subject-based schema.	Reported / reconcile version
TC-04	Attempt overlapping room schedule.	Conflicting schedule is rejected with a validation message.	User reported overlap validation; source includes overlap query. Save the rejection screenshot for final evidence.	Reported pass
TC-05	Enter marks and view result.	Percentage, grade and status appear in result register.	Screenshot shows marks/results interface; upper-bound DB enforcement and grade-rule consistency need verification.	Partially evidenced
TC-06	Export report to CSV.	CSV download contains the displayed report rows and headings.	User reported exports tested; reports screen screenshot supplied.	Reported pass
TC-07	Duplicate roll number or registration.	Database rejects duplicate and UI displays understandable error.	UNIQUE constraints and exception handler exist; negative test output not supplied.	Pending evidence
TC-08	Schedule outside session dates.	Schedule is rejected.	Validation exists in packaged app.py; no actual negative-test output supplied.	Pending evidence
TC-09	Room capacity exceeded.	Schedule is rejected if registered count exceeds capacity.	Validation exists in packaged app.py; no actual negative-test output supplied.	Pending evidence
TC-10	Enter marks above maximum using non-UI SQL.	Database should reject invalid marks.	Current table CHECK only enforces marks >= 0; upper bound is enforced in the UI control only.	Gap identified
TC-11	Run destructive SQL in custom explorer.	Non-read-only statements must not modify data.	UI checks statement prefix only; a stronger read-only mechanism is required.	Security gap identified
TC-12	Clean clone install and launch.	README setup succeeds from a fresh environment.	Not independently run while preparing this report.	Pending

Testing conclusion: the available evidence supports the existence of the main interface and some user-reported workflows. It does not establish that every constraint, delete path, clean installation and security edge case has been independently verified. Final submission should include actual positive and negative test outputs from the exact repository version.

CHAPTER 4

CONCLUSION

ExamFlow demonstrates how a relational database can support an examination workflow that spans master data, student registrations, timetable scheduling, marks entry and reporting. The prototype separates major entities into linked relations, uses primary and foreign keys to preserve references, and uses uniqueness and check constraints for selected data-quality rules. Python and Streamlit provide an accessible interface, while SQLite stores local records and supports SQL-based reports. The result_view derives percentage, grade and pass/fail status from the stored registration and marks data.

The project meets the intended academic objective of connecting a database design to a working application prototype and presenting evidence of its main screens. However, the supplied presentation screenshots and the app.py in the prototype ZIP appear to represent different iterations. The schema.sql file is also a stub rather than the full standalone DDL. These consistency issues should be resolved before final submission so the report, ER diagram, code and screenshots all describe the same build.

4.1 Limitations and Future Enhancements

- Reconcile the screenshot/presentation schema (Subjects, Exams, Results, section and publication status) with the packaged implementation schema (Courses, Exam Sessions, Registrations, Schedules and Marks), then recapture screenshots from the final build.
- Populate schema.sql with complete executable DDL or explicitly designate app.py:init_db() as the only schema source.
- Use grade_rules as the source of grade boundaries instead of hard-coded CASE conditions; validate that grade ranges do not overlap or leave gaps.
- Enforce marks_obtained <= course.max_marks at the database layer as well as in the UI.
- Strengthen the custom SQL explorer with a read-only database connection or SQLite authorizer; checking only the first word is insufficient for security.
- Add authentication, role-based permissions, audit logs, backups, input sanitization, transaction handling and production deployment controls before any real institutional use.
- Expand automated tests for duplicate records, invalid dates/times, capacity violations, invigilator conflicts, mark boundaries, foreign-key violations and CSV content.
- Add indexes based on measured query workload and inspect query plans before claiming performance improvements.

REFERENCES

References below point to official technology documentation relevant to the implementation. Access date: 15 October 2026.

[1] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python Documentation. [Online]. Available: https://docs.python.org/3/library/sqlite3.html. Accessed: Oct. 15, 2026.

[2] SQLite Consortium, “CREATE TABLE,” SQLite Documentation. [Online]. Available: https://www.sqlite.org/lang_createtable.html. Accessed: Oct. 15, 2026.

[3] SQLite Consortium, “CREATE VIEW,” SQLite Documentation. [Online]. Available: https://www.sqlite.org/lang_createview.html. Accessed: Oct. 15, 2026.

[4] Streamlit, Inc., “Streamlit documentation,” Streamlit Documentation. [Online]. Available: https://docs.streamlit.io/. Accessed: Oct. 15, 2026.

[5] pandas development team, “pandas.DataFrame.to_csv,” pandas Documentation. [Online]. Available: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html. Accessed: Oct. 15, 2026.

[6] K. Levin Josewell, “Examination Scheduling and Result Processing System (ExamFlow),” source-code repository. [Online]. Available: https://github.com/levinjosewell/examination-scheduling-result-processing-system. Accessed: Oct. 15, 2026.

APPENDIX A — COMPLETE DATABASE SCRIPT REFERENCE

The complete schema definition used by the provided package is inside app.py in the init_db() function. The separate schema.sql file in the uploaded ZIP contains only a comment and is not a complete executable script. For a clean academic hand-in, copy the CREATE TABLE and CREATE VIEW statements from init_db() into schema.sql, test them against a new empty SQLite database, and update this appendix to reference that verified file.

```
# Run the application (creates exam_system.db on first launch)
python -m pip install -r requirements.txt
streamlit run app.py

# Main schema source in the supplied package:
# app.py -> init_db() -> CREATE TABLE ... and CREATE VIEW result_view ...
```

APPENDIX B — APPLICATION SOURCE CODE

Repository: https://github.com/levinjosewell/examination-scheduling-result-processing-system

Main files identified in the supplied ZIP: app.py, requirements.txt, README.md, schema.sql, .gitignore and documentation/VIDEO_LINK_TEMPLATE.md (folder layout may vary between the local ZIP and GitHub). Confirm the final repository contains the reviewed report, final presentation decks, test plan and evidence before submission.

APPENDIX C — SAMPLE DATA AND OUTPUTS

The packaged application seeds a small fictional dataset when the database is empty. Example records include programme labels such as B.Tech AI & ML and B.Tech CSE, student and course identifiers, faculty, rooms, an exam session, registrations and marks. The screenshots supplied separately show a larger synthetic dataset with dashboard figures including 1,200 students, 60 faculty, 180 subjects, 264 scheduled exams and 18,264 result entries. Those figures are screenshot-reported demo values and must not be described as live university records or as counts generated by the small seed block in the uploaded ZIP.

Additional evidence to attach before submission: (1) a successful clean launch, (2) a newly created record persisting after refresh, (3) rejected duplicate registration, (4) rejected overlapping room/invigilator schedule, (5) rejected out-of-session date, (6) marks boundary test, and (7) CSV export opened and checked. Label each item with the test ID used in Section 3.8.